eNMRIS Database Schema and API Contract

Version: 1.0 Status: Agreed design baseline Date: 9 November 2026 Scope: Prototype medicines register for the Medicines Regulatory Authority (MRA), Eswatini

1. Purpose and design decisions

This contract establishes the data model, approval workflow, and application/FHIR interfaces before feature implementation. It reflects the project plan and the decisions confirmed by the team. openapi.yaml is the machine-readable description of the HTTP contract.

- MySQL 8, InnoDB, and utf8mb4; application tables use snake_case.
- CMS product codes are strings, never numbers, to preserve identifiers exactly.
- The CMS workbook is the source of truth for product-template columns. It contains 19 product rows and 18 unique programProducts records. Two trailing rows in that sheet are programme-name notes, not records; the plan's figure of 18 programme links is correct.
- Registration status changes and recall issuance/closure require approval by a different person from the requester.
- If an Admin requests a regulatory change, a specifically authorized Regulatory Officer may approve it. The permission is limited to regulatory approvals and does not grant user or system administration.
- The API separates public, limited medicine verification from authenticated staff and FHIR interfaces.
- The application does not delete regulatory records; corrections and status changes remain traceable.

2. Roles and approval controls

Roles use stable role_code values stored in roles, with editable display_label values. Actions are granted through role_permissions; authorization logic and clients use role/permission codes, never display labels. Account roles and Board positions are role rows, and effective permissions combine the user's account role, optional position role, and explicit user grants for other scoped capabilities. Governance actions are granted only through role_permissions.

- Admin: manages configuration and may manage regulatory records or approve an Officer's regulatory change request. Admin replacement is invitation-only; Admins cannot create public Admin accounts.
- Admin tester (ADMIN_TESTER): a temporary, manually assigned test role with Admin-equivalent invitation permissions. It cannot approve, is excluded from active-Admin counts, and is deactivated with all sessions revoked by the production startup safeguard. Startup warns while one is active. Conversion and cleanup are separate audited maintenance operations, not part of replacement sign-in.
- Head of IT (HEAD_OF_IT): a locally bootstrapped role that can only initiate Board Officer replacement requests. It has no Admin or Officer authority and cannot decide a replacement.
- Regulatory Officer: manages regulatory records but not users or system configuration.
- Board Chair / Board Vice Chair: special Officer positions bootstrapped locally and independently. Each enrolls an authenticator at first sign-in. Either enrolled Board Officer may approve or reject an Admin replacement invitation; this does not grant general Admin privileges.
- Board Officer replacement: an active Head of IT names one outgoing Officer position and an incoming person. The current non-outgoing Officer is saved as the only approver and must decide with a fresh TOTP code and reason. The request expires in seven days. Only one uncompleted replacement may be open at a time, keeping the saved approver available. Approval creates a hash-only, single-use 24-hour setup link. The outgoing Officer stays active until the incoming Officer enrolls MFA and completes a successful sign-in; the position handoff and outgoing deactivation are atomic. If there is no continuing Officer, use the separate audited local emergency bootstrap. Admin and ADMIN_TESTER cannot start, approve, reject, or alter these replacements.

- Designated approving Officer: an Officer granted the approve_regulatory_changes permission. This Officer may review and approve an Admin's regulatory change request, but gains no administrative privileges.
- Public visitor: may search only the approved-medicine and active-recall fields exposed by the public endpoint.
- FHIR client: an individually registered external system with read-only scopes and revocable credentials.

The requester and approver must always be different users. An active Board Officer may approve an Admin replacement; only the saved continuing, non-outgoing Board Officer may approve an Officer replacement. Admin and ADMIN_TESTER accounts cannot initiate or decide Officer replacements, and the Head of IT, outgoing Officer, and incoming person cannot decide them. If no continuing Officer exists, use the separate audited local emergency bootstrap rather than bypassing approval in the application.

Every approval or rejection records the requester, approver, reason, decision, and timestamps. The server obtains the actor identity from the authenticated session; clients cannot submit an approved_by value.

An Admin replacement request expires if no Board Officer decides within seven days. Approval creates a 24-hour setup link delivered by configured SMTP; the token is single-use, hash-only at rest, and is never returned by an API or logged. The invited person enrolls a password and authenticator before an account is created. A successful test sign-in is audited; it does not deactivate the previous Admin. Only after the test sign-in may the separate manual conversion script move the previous account to ADMIN_TESTER. A second manual cleanup script deactivates the tester only when the verified replacement is the sole active Admin. Accounts are deactivated, not deleted.

3. Database schema

3.1 Conventions and constraints

- Use UUID primary keys for application records and UTC timestamps.
- Include created_at, updated_at, created_by, and updated_by on mutable business records where applicable. Audit events are append-only and do not have updated_at.
- Use foreign keys, unique constraints, checks, and indexes for relationships and lookup fields.
- Apply all writes to a business record, its history, and its audit event in one database transaction.
- Use a unique registration number, subject to confirmation against the MRA's official numbering rules.
- Enforce at most one non-terminal registration for a given product and holder. PENDING, REGISTERED, and SUSPENDED are non-terminal; REVOKED and EXPIRED are terminal. In MySQL, this can be implemented with a generated nullable active-slot column and a unique index on (product_id, holder_id, active_slot).
- A new registration starts as PENDING. Every later status transition requires an approved request. When its expiry date is reached, the system transitions an eligible registration to EXPIRED and records a SYSTEM history and audit event.

3.2 Tables

- products ? id, unique product_code (VARCHAR), dispensable, name, description, pack_size, pack_rounding_threshold, round_to_zero, and common timestamps/actors. Preserve the CMS workbook's source values.
- product_programmes ? id, product_id, program, doses_per_patient, active, category, full_supply, display_order, price_per_pack; unique on (product_id, program). Import all 18 data rows and skip the two explanatory legend rows.
- organisations ? id, name, org_type (at least HOLDER, MANUFACTURER), and ISO alpha-3 country_code.
- registrations ? id, product_id, holder_id, unique registration_number, status, approval_date, expiry_date, and status_reason.
- registration_status_history ? id, registration_id, old_status, new_status, reason, changed_by, approved_by (nullable for system events), and changed_at.

- registration_status_change_requests ? id, registration_id, requested_status, reason, requested_by, decision (PENDING, APPROVED, REJECTED), decided_by, requested_at, and decided_at.
- batches ? id, product_id, batch_code, manufacture_date, and expiry_date; unique on (product_id, batch_code).
- recalls ? id, recall_class (I, II, III), reason, status (ACTIVE, CLOSED), issued_by, issued_at, closed_by, and closed_at.
- recall_batches ? recall_id, batch_id; supports one recall covering multiple batches and a batch having recall history.
- recall_change_requests ? id, request type (ISSUE, CLOSE), optional recall_id, class/reason/target status as applicable, requester, decision, approver, and timestamps. An associated request-batch table records the batches for a proposed recall.
- recall_status_history ? id, recall_id, old/new status, reason, requester, approver, linked change request, and timestamp.
- roles / role_permissions ? stable role codes, editable display labels and position marker; action grants keyed by role code and permission code.
- app_users ? id, name, unique email, password_hash, foreign-key role_code, optional unique officer_position role code, encrypted time-based one-time password (TOTP) secret, last accepted TOTP step, hash-only first-sign-in enrollment challenge and expiry, MFA state, and active.
- app_user_permissions ? grants scoped permissions such as approve_regulatory_changes to designated Officers; record who granted/revoked each permission.
- app_sessions ? user, hash of the opaque session token, creation/last-activity/expiry times, and revocation time.
- admin_invitations ? replacement Admin requester, invitee, replaced Admin, approval/rejection actor and reason, lifecycle timestamps, delivery state, and only a hash of the short-lived single-use setup token.
- officer_replacement_requests ? Head-of-IT or emergency replacement request, outgoing and continuing Officers, incoming identity, decision and reason, lifecycle and mail delivery timestamps, and only a hash of the short-lived single-use setup token.
- auth_login_attempts ? keyed login failure counters and temporary lock timestamps.
- api_clients, api_client_scopes, and api_access_tokens ? client identifier, hashed secret, read-only scopes, hashed short-lived bearer tokens, active/expiry/revocation state, and timestamps.
- audit_log ? append-only event ID, actor type and ID (USER, CLIENT, or SYSTEM), actor role code and display-label snapshot, action, entity/table, record ID, before/after JSON, reason, timestamp, and request ID.

The application database account must not have UPDATE or DELETE permission on audit_log. Database administration access must be restricted and audited separately.

4. Application API

Base path: /api/v1. All staff routes require authentication and authorization. Mutations validate input on the server and use parameterized SQL.

4.1 Authentication and staff

- POST /auth/login ? email, password, and time-based one-time password (TOTP) code; creates a secure session.
- POST /auth/mfa/enrollment/complete ? completes a password-authenticated Board Officer's short-lived first-sign-in enrollment challenge; stores the challenge hash-only, encrypts the TOTP secret, and creates the first session only after code verification.
- POST /auth/logout ? revokes the current session.

- GET /auth/me ? returns the authenticated user's stable roleCode, editable roleLabel, optional officerPosition and officerPositionLabel, and effective permissions.
- GET /admin-invitations ? lists the caller's requests, or the Board Officers' review queue.
- POST /admin-invitations ? Admin or ADMIN_TESTER submits { "name": "...", "email": "..." }; email must match an exact configured allowlisted domain. This creates a pending invitation, not an account. Testers cannot approve.
- POST /admin-invitations/{invitationId}/decision ? Board Chair or Vice Chair submits { "decision": "APPROVE" | "REJECT", "reason": "...", "totpCode": "000000" }. Either Officer can decide independently; self-approval is blocked.
- POST /admin-invitations/{invitationId}/reissue ? the approving Officer may reissue a failed or expired setup link with a reason and a fresh authenticator code. Reissue invalidates the prior token.
- POST /admin-invitations/setup/inspect ? public check for the single-use setup token; body contains the token. Returns only invitee name/email and expiry, never the token.
- POST /admin-invitations/setup/complete ? public single-use enrollment; body contains the token, password, TOTP secret, and current code. The API stores only a token hash and encrypts the authenticator secret.
- GET /officer-replacements ? users with start_replacement see their requests; users with approve_replacement see the review queue. Returns both requests and active Board-position codes with editable display labels. Expired approval requests are marked and audited.
- POST /officer-replacements ? active Head of IT submits { "outgoingPosition": "BOARD_CHAIR" | "BOARD_VICE_CHAIR", "incomingName": "...", "incomingEmail": "..." }. Both current Officers receive notices. The incoming email must match the configured allowlist.
- POST /officer-replacements/{requestId}/decision ? only the saved continuing Officer submits { "decision": "APPROVE" | "REJECT", "reason": "...", "totpCode": "000000" }.
- POST /officer-replacements/{requestId}/reissue ? continuing Officer reissues a failed or expired setup link with reason and fresh authenticator code.
- POST /officer-replacements/setup/inspect and /officer-replacements/setup/complete ? public single-use incoming Officer account setup. Account creation does not hand over the position until successful sign-in.
- GET /approvals>=status=pending ? lists regulatory requests the caller is permitted to review.
- POST /approvals/{requestId}/approve and POST /approvals/{requestId}/reject ? record a decision and reason; requester cannot decide their own request.

4.2 Register and regulatory records

- GET /organisations>=q=... and POST /organisations ? search/create holders and manufacturers.
- GET /products>=q={term}&page={n}&pageSize={n} and GET /products/{id} ? staff product search/read.
- POST /products/import and GET /products/export ? import/export the CMS workbook layout; validate the workbook before committing changes.
- POST /registrations ? creates a PENDING registration and its initial history/audit event.
- GET /registrations/{id} and GET /registrations/{id}/status-history ? read a registration and its history.
- POST /registrations/{id}/status-change-requests ? body: { "targetStatus": "...", "reason": "..." }.
- POST /batches and GET /batches/{id} ? create/read batches.
- POST /recall-change-requests ? propose an ISSUE with class, reason, and one or more batch IDs.
- POST /recalls/{id}/closure-requests ? propose closing a recall with a reason.
- Only an approved request changes a registration or recall's effective status. Approval, status update, history, and audit are committed atomically.

4.3 Public verification

- GET /public/search>=q={term} ? no login; returns only limited product details for approved medicines and active recalls. Do not expose staff accounts, private holder data, internal history, or audit details.

4.4 FHIR R4

- GET /fhir/metadata ? authenticated client capability statement.
- GET /fhir/Medication>=code={cmsProductCode} and GET /fhir/Medication/{id} ? authenticated, read-only search/read.
- Authenticate each system independently using revocable client credentials and a short-lived bearer token with read-only FHIR scope.
- Map CMS product code to a documented identifier system; map batches to Medication.batch. Document registration authorization and recall mappings; use a documented extension or other suitable R4 representation for recalls, which have no dedicated R4 resource.

4.5 Response conventions

- Success: { "data": ..., "meta": ... }; list endpoints use bounded pagination.
- Error: { "error": { "code": "...", "message": "...", "details": ... }, "requestId": "..." }.
- Use appropriate HTTP status codes. Never return stack traces, SQL, secrets, time-based one-time password (TOTP) values, or password hashes.

5. Security requirements

- RBAC: enforce the role and scoped approval permission on every protected route and FHIR scope.
- MFA: require a password plus an authenticator-app time-based one-time password (TOTP) for staff in the MVP. This method is not phishing-resistant; passkeys/WebAuthn are recommended before production.
- Transport: enforce TLS and HSTS in production. Certificate pinning is not applicable to the current web application; reconsider only if a native mobile app is built.
- Input/database: validate request bodies, parameters, uploaded files, and search terms; use parameterized queries exclusively.
- Login protection: rate-limit after 5 failed attempts per account/IP in 15 minutes; apply temporary backoff rather than permanent lockout.
- Sessions: use a high-entropy opaque session token in a Secure, HttpOnly, SameSite cookie; store only its hash, support revocation, expire after 20 minutes idle, and protect cookie-authenticated mutations against CSRF. Do not bind sessions to browser/device fingerprints.
- Passwords: recommended baseline is at least 12 characters, permit passphrases, reject commonly compromised passwords, and do not force periodic changes absent compromise.
- Audit: record authentication/security events and all regulatory mutations. For Admin handover, audit invitation, approval/rejection and Officer role/position, token issue/use, email success/failure, test sign-in, tester conversion, and tester deactivation. For Officer replacement, audit requester, notices sent/failed, approval/rejection and deciding Officer, token issue/use, email sent/failed, successful test sign-in, outgoing deactivation, and emergency use, all with timestamps. Keep the audit log append-only to the application, and restrict direct database access.
- Email accounts: require ALLOWED_EMAIL_DOMAINS to be explicitly configured. Match exact domains only; do not allow subdomains or lookalikes implicitly. Revalidate at account creation and before sending. Development-only email-domain and test-inbox overrides must never be enabled in production.

6. Contract notes

- The workbook is available as CMS Products Template.xlsx; its actual columns are authoritative for import/export. The programProducts sheet has 18 data records followed by two legend rows, which are not seed records.
- database/01_schema.sql defines the MySQL tables; database/02_seed.sql loads the 19 products and 18 programme mappings. For existing databases, apply migrations 04, 05, 06, then database/07_roles_and_role_permissions.sql once. The final migration converts enum-based account and position roles to role-code foreign keys and snapshots audit actor labels. openapi.yaml describes the application and FHIR endpoints.
- The MRA's registration-number rule and official registration data have not yet been supplied. Keep the numbering format configurable and use test data until approved data is provided.
- database/04_admin_invitation_workflow.sql upgrades an existing authentication database; run it once, followed by database/05_admin_tester_and_officer_mfa.sql, database/06_officer_replacement_workflow.sql, and database/07_roles_and_role_permissions.sql once each and in order before restarting the updated backend. npm run officer:bootstrap prompts for one Officer's password; authenticator enrollment occurs at first sign-in.
- npm run staff:bootstrap -- --role HEAD_OF_IT creates Head of IT with a hidden interactive password prompt. npm run officer:bootstrap remains the normal Board Officer bootstrap; npm run officer:bootstrap -- --emergency-replace is the separately invoked emergency path when no continuing Officer can decide. It stages the incoming Officer and preserves the outgoing account until test sign-in. Fresh installations get the role model and workflows from database/01_schema.sql.
- Ethereal development mail uses Nodemailer with SMTP_HOST=smtp.ethereal.email, SMTP_PORT=587, and private SMTP_USER/SMTP_PASS values. Set the temporary allowlist to ethereal.email; the recipient address appears in the Ethereal account's Messages page. Replace test domains and SMTP credentials before production.
- The manual npm run admin-tester:convert and npm run admin-tester:deactivate commands prompt for a maintenance MySQL account, hidden password, target, operator, reason, and exact confirmation. They preserve the original credentials on conversion, never delete accounts, and audit all changes. Conversion requires the verified replacement to be the only other active Admin; cleanup requires that replacement to be the sole active Admin. Both commands refuse production mode.
- During initial MySQL setup, an account lookup returning 0 rows means the configured eNMRIS MySQL user does not exist and explains ER_ACCESS_DENIED_ERROR. Create the host-specific localhost and 127.0.0.1 accounts, grant only the documented table/column permissions, rerun the lookup to confirm both rows, then restart the backend and verify /health.
- Live integration with CMS/eLMIS and mobile applications remain out of scope. The FHIR interface is read-only and designed for future authorized systems.